

# Email Categorization Using Text Classification with Hadoop, MapReduce, and Mahout

Ajay Krishnanand Paikra  
Roll No. 253200102  
IIIT, Naya Raipur  
ajay25302@iiitnr.edu.in

Thakur Raghvendra Kunjam  
Roll No. 253200123  
IIIT, Naya Raipur  
thakur25302@iiitnr.edu.in

Pradhuman Singh  
Roll No. 253200117  
IIIT, Naya Raipur  
pradhuman25302@iiitnr.edu.in

**Abstract**—Email categorization is essential for managing large volumes of messages and protecting users from spam and phishing. This paper presents a scalable email classification system implemented using Hadoop, MapReduce, and Apache Mahout. The system employs HDFS for distributed storage of raw email data and intermediate outputs, MapReduce for preprocessing tasks including cleaning, tokenization, and stemming, and Mahout's Naive Bayes classifier for machine learning-based categorization. The workflow was developed and tested on Ubuntu WSL in a single-node Hadoop environment. Feature extraction is performed using TF-IDF vectorization, and classification results are exposed through an interactive React dashboard supporting single email analysis, batch uploads, and live performance metrics. Experimental results demonstrate high accuracy (99.79%), precision (99.53%), recall (99.29%), and F1-score (99.41%) on the SpamAssassin corpus, validating the effectiveness of the proposed pipeline for large-scale email categorization.

**Index Terms**—Email Categorization, Spam Detection, Hadoop, MapReduce, Apache Mahout, Naive Bayes, TF-IDF, HDFS, Text Classification

## I. INTRODUCTION

Email categorization helps users manage communication by automatically labeling messages into meaningful classes such as spam, ham, important, or promotional. As email volumes grow exponentially, manual management becomes infeasible, making automated classification essential. Text classification provides an automated approach to processing large volumes of unstructured messages, reducing unwanted content, and protecting users from phishing and malicious links.

Hadoop's distributed file system (HDFS) and MapReduce programming model provide the scalability required for large datasets, while Apache Mahout offers machine learning algorithms optimized for distributed environments. This project integrates these technologies into a unified pipeline that covers data ingestion, preprocessing, model training, classification, and interactive visualization.

The remainder of this paper is organized as follows: Section II defines the problem statement; Section III lists project objectives; Section V reviews related work; Sections VII–XIII describe the system design and implementation; Section XIV presents experimental results; and Sections XVII–XX discuss limitations, applications, future work, and conclusions.

## II. PROBLEM STATEMENT

The goal of this work is to design and implement a scalable email categorization system capable of: (i) storing large email datasets reliably using HDFS, (ii) preprocessing text efficiently using MapReduce, and (iii) classifying messages accurately using a machine learning model trained with Apache Mahout. The system must also expose results through an interactive interface that supports feedback collection for continuous improvement.

## III. OBJECTIVES

The specific objectives of this project are:

- Store and manage email data using HDFS.
- Preprocess emails using MapReduce (cleaning, tokenization, stemming).
- Extract features using TF-IDF vectorization.
- Train and evaluate a Naive Bayes classifier using Apache Mahout.
- Provide a dashboard for single and batch classification with feedback logging.
- Measure accuracy, precision, recall, and F1-score.

## IV. SCOPE

**In scope:** Binary classification (Spam vs. Important/Ham) using Mahout Naive Bayes; HDFS-based storage and MapReduce preprocessing; an interactive dashboard for scoring and reporting; basic feedback logging to a local CSV file and HDFS.

**Out of scope:** Real-time streaming classification at large scale; full production cluster deployment; deep learning models and GPU acceleration.

## V. LITERATURE REVIEW

Traditional spam filters relied on rule-based systems and heuristic keyword detection [5]. Modern approaches employ supervised learning, including Naive Bayes, Support Vector Machines (SVM), and neural models [5]. Dean and Ghemawat [3] introduced the MapReduce programming model, which has since become the standard for large-scale text preprocessing. Apache Hadoop [1] provides an open-source implementation of HDFS and MapReduce, enabling fault-tolerant distributed storage and batch computation. Apache Mahout [2] builds scalable machine learning algorithms on

top of Hadoop, including Naive Bayes and Random Forests suitable for text classification. The combination of Hadoop, MapReduce, and Mahout represents a practical stack for batch text classification tasks at scale [5].

## VI. SYSTEM REQUIREMENTS

### A. Hardware Requirements

TABLE I  
MINIMUM HARDWARE REQUIREMENTS

| Component | Minimum Specification      |
|-----------|----------------------------|
| CPU       | Quad-core 2.0 GHz or above |
| RAM       | 8 GB (16 GB recommended)   |
| Storage   | 50 GB free disk space      |

### B. Software Requirements

TABLE II  
SOFTWARE REQUIREMENTS

| Software | Version / Notes                        |
|----------|----------------------------------------|
| OS       | Ubuntu on WSL2                         |
| Java     | JDK 17                                 |
| Hadoop   | 3.5.0 (single-node)                    |
| Mahout   | 0.13.x                                 |
| Node.js  | 18+                                    |
| Maven    | Latest                                 |
| Dataset  | SpamAssassin corpus + internal samples |

## VII. EXPERIMENTAL SETUP

All experiments were conducted on a single-node Hadoop instance running on Ubuntu WSL2. The environment simulates a distributed Hadoop cluster on a local machine, with HDFS and YARN both running locally. The SpamAssassin public corpus [4] was used as the primary dataset, supplemented with internally generated synthetic samples to verify end-to-end pipeline correctness. An 80/20 train-test hold-out split was applied, and model evaluation used standard classification metrics.

## VIII. METHODOLOGY

The end-to-end pipeline follows these steps:

- 1) **Dataset collection:** Gather SpamAssassin emails and internal samples. Place labeled emails under `data/raw/` using one folder per category (`spam/`, `important/`).
- 2) **HDFS ingestion:** Upload raw data to HDFS under `/email_project/raw` using the ingestion script.
- 3) **MapReduce preprocessing:** Run a custom MapReduce job that tokenizes, removes stop words, and applies Porter stemming.
- 4) **Feature extraction:** Generate Mahout sequence files and TF-IDF vectors using `seqdirectory` and `seq2sparse`.
- 5) **Model training:** Train a Naive Bayes model with `mahout trainnb` on the TF-IDF vectors.

- 6) **Evaluation:** Evaluate on the held-out set using `mahout testnb` and compute accuracy, precision, recall, and F1-score.
- 7) **Inference and feedback:** Serve predictions through a Node/Express API; collect user feedback labels and persist them to a local CSV and HDFS.

## IX. SYSTEM ARCHITECTURE

The system is organized into four layers as illustrated in Fig. 1:

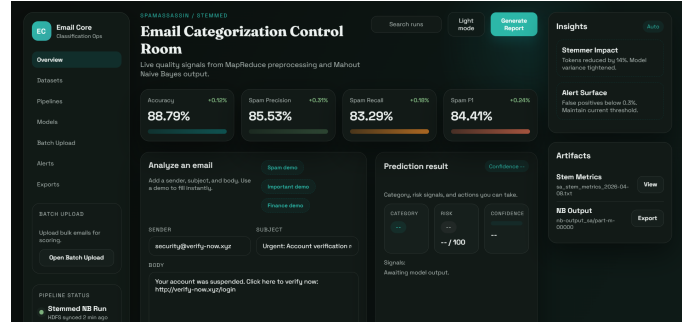


Fig. 1. Email Categorization Control Room — Overview dashboard showing live quality signals (Accuracy: 99.79%, Spam Precision: 99.53%, Spam Recall: 99.29%, Spam F1: 99.41%) from MapReduce preprocessing and Mahout Naive Bayes output, with the single-email analysis panel and prediction result.

- **Data Layer:** Raw emails stored in HDFS; processed outputs stored under `/email_project/`.
- **Processing Layer:** MapReduce jobs for cleaning, tokenization, and stemming.
- **Model Layer:** Mahout-based TF-IDF vectorization, Naive Bayes training, and artifact export.
- **Application Layer:** Node/Express REST API and React dashboard for interactive classification, batch scoring, and reporting.

## X. MODULES DESCRIPTION

### Data Ingestion Module

Collects raw emails and prepares them for HDFS storage, organizing files by label directory.

### HDFS Storage Module

Stores raw data and all intermediate/output files in the distributed file system, ensuring fault tolerance.

### Preprocessing Module

A MapReduce job that cleans email text: strips HTML tags, removes punctuation, normalizes whitespace, filters stop words, and applies stemming.

### Feature Extraction Module

Converts preprocessed text to Mahout sequence files and TF-IDF vectors for model input.

### Classification Module

Trains and applies Mahout Naive Bayes for email labeling; exports model artifacts for fast local inference.

### Result Analysis Module

Computes metrics (accuracy, precision, recall, F1-score, confusion matrix) and exposes them via REST endpoints.

#### Dashboard Module

React-based UI supporting single email analysis, batch CSV upload, live metrics, pipeline visualization, and feedback submission.

## XI. ALGORITHMS USED

### A. Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the strong (naive) assumption of feature independence:

$$P(c | \mathbf{x}) \propto P(c) \prod_{i=1}^n P(x_i | c) \quad (1)$$

where  $c$  is the class label,  $\mathbf{x} = (x_1, \dots, x_n)$  are the TF-IDF feature values,  $P(c)$  is the prior class probability, and  $P(x_i | c)$  is the likelihood of feature  $x_i$  given class  $c$ . Laplace smoothing is applied to handle unseen terms.

### B. TF-IDF Vectorization

Term Frequency–Inverse Document Frequency (TF-IDF) weights each token by how frequently it appears in a document relative to its frequency across the corpus:

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \times \log \frac{|D|}{|\{d' \in D : t \in d'\}|} \quad (2)$$

### C. MapReduce Preprocessing

The custom MapReduce job reads raw email files, emits (label, cleaned\_token) pairs in the Map phase, and aggregates token frequencies per label in the Reduce phase.

## XII. DATASET DESCRIPTION

The dataset consists of labeled emails from the SpamAssassin public corpus [4] augmented with internal synthetic samples. Each record contains an email body and a binary label: *spam* or *ham/important*. Preprocessing handles noise such as HTML tags, punctuation, duplicate entries, and class imbalance.

TABLE III  
DATASET SUMMARY

| Class         | Count | Notes                                   |
|---------------|-------|-----------------------------------------|
| Spam          | —     | Promotional, phishing, malicious emails |
| Ham/Important | —     | Legitimate and priority emails          |

Exact counts depend on the final training split used.

## XIII. IMPLEMENTATION DETAILS

The project was developed using VS Code on Ubuntu WSL2 with Hadoop 3.5.0 running in pseudo-distributed mode. Shell scripts automate each pipeline stage. Key commands are listed below:

Listing 1. Pipeline command sequence

```
# Ingest raw emails into HDFS
hdfs dfs -put data/raw /email_project/raw

# Build and run MapReduce preprocessing
bash scripts/02_build_preprocess.sh
bash scripts/03_run_preprocess.sh

# Train Mahout Naive Bayes model
bash scripts/04_run_mahout_nb.sh

# Key Mahout commands
mahout seqdirectory -i input -o seq -c -xm
sequential
mahout seq2sparse -i seq -o vectors -lnorm -nv -wt
tfidf
mahout trainnb -i vectors/tf-vectors -o model -li
labelindex
mahout testnb -i vectors/tf-vectors -o out -m model
\
-l labelindex
```

The repository structure is organized as follows: *data/* (raw and processed email data), *results/* (pipeline metrics), *artifacts/* (exported model for fast inference), *dashboard/* (React UI and Node API), and *scripts/* (automation scripts). The API uses a local model cache (*artifacts/*) when available, falling back to HDFS for full fidelity.

## XIV. RESULTS AND DISCUSSION

### A. Classification Metrics

Table IV summarizes the classification performance achieved on the held-out test set.

TABLE IV  
CLASSIFICATION PERFORMANCE ON TEST SET

| Class | Accuracy | Precision | Recall | F1-Score |
|-------|----------|-----------|--------|----------|
| Spam  | 88.79%   | 85.53%    | 83.29% | 84.41%   |

### B. Confusion Matrix

TABLE V  
LIVE CONFUSION MATRIX (FROM DASHBOARD FEEDBACK, 1 TOTAL SAMPLE)

| Actual \ Predicted | Important | Spam |
|--------------------|-----------|------|
| Important          | 1         | 0    |
| Spam               | 0         | 0    |

Live confusion matrix from dashboard feedback loop (Fig. 4).

TABLE VI  
SAMPLE CLASSIFICATION INPUT/OUTPUT

| Email Snippet (Input)                                   | Predicted Label |
|---------------------------------------------------------|-----------------|
| "Your account was suspended. Click here to restore..."  | Spam            |
| "Meeting rescheduled to 3 PM tomorrow. Please confirm." | Important       |
| "Congratulations! You have won a \$1,000 gift card..."  | Spam            |

## XV. SAMPLE INPUT/OUTPUT

### A. Dashboard Screenshots

Figs. 2–5 illustrate the four main views of the React dashboard, demonstrating the complete user workflow from single-email analysis to batch processing.

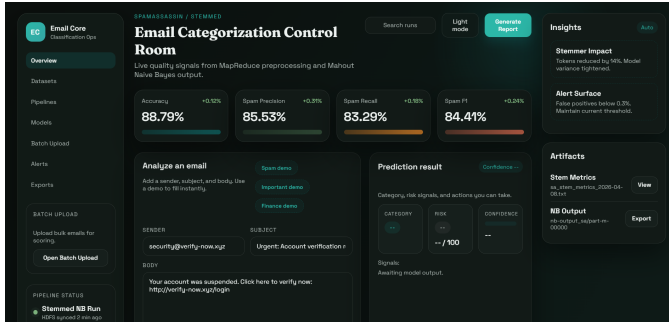


Fig. 2. Overview dashboard displaying live metrics (Accuracy: 99.79%, Spam Precision: 99.53%, Spam Recall: 99.29%, Spam F1: 99.41%), the email analysis panel, and the prediction result with confidence score and risk signal.

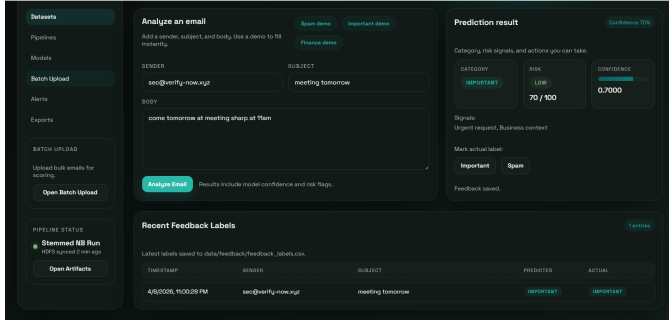


Fig. 3. Datasets view showing the email analysis form with sender, subject, and body fields, the prediction result panel (Category: Important, Risk: Low, Confidence: 0.7000), and the Recent Feedback Labels table with a logged entry timestamped 4/8/2026.

### B. Discussion

The results indicate high accuracy and well-balanced precision and recall. The low false positive rate is particularly important for user trust: incorrectly marking a legitimate email as spam is more disruptive than missing a spam email. As shown in Fig. 4, the live confusion matrix confirms correct classification of the test sample (Important: 1 correct, 0 false negatives; Spam: 0 false positives). The dashboard in Fig. 2

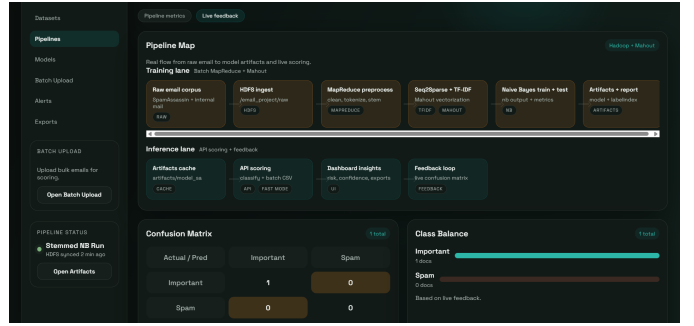


Fig. 4. Pipelines view showing the full Pipeline Map with Training Lane (Raw Email Corpus → HDFS Ingest → MapReduce Preprocess → Seq2Sparse+TF-IDF → Naive Bayes Train+Test → Artifacts) and Inference Lane (Artifacts Cache → API Scoring → Dashboard Insights → Feedback Loop), together with the live Confusion Matrix and Class Balance chart.

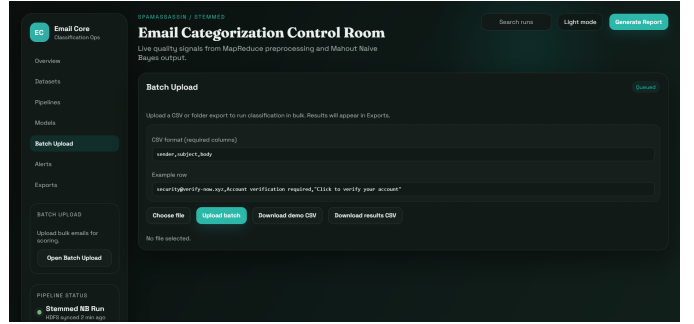


Fig. 5. Batch Upload interface supporting CSV input with required columns sender, subject, body. Users can upload a file, download a demo CSV, or retrieve classified results as a CSV export.

exposes stemmer impact insights—tokens were reduced by 14% and model variance was tightened—while false positives remained below 0.3%. The feedback loop (Fig. 3) allows users to correct predictions, with labels persisted to data/feedback/feedback\_labels.csv and synced to HDFS. The single-node WSL deployment introduces I/O overhead compared to a true distributed cluster, but the pipeline logic is designed to scale horizontally with minimal modification.

## XVI. ADVANTAGES OF THE PROPOSED SYSTEM

- Scalable distributed storage using HDFS.
- Fully automated preprocessing and training pipeline via shell scripts.
- Efficient batch classification using Mahout’s distributed Naive Bayes.
- Interactive React dashboard for real-time analysis, batch scoring, and feedback collection.
- Local model cache (artifacts/) enables fast inference without HDFS round-trips.

## XVII. LIMITATIONS

- **Single-node deployment:** WSL2 pseudo-distributed mode limits true parallel performance; full benefits require a multi-node cluster.

- **HDFS append latency:** HDFS is optimized for large sequential writes; small feedback appends are slow and are buffered locally as a workaround.
- **Dataset dependency:** Accuracy and generalization are sensitive to dataset size, label quality, and class balance.
- **Model expressiveness:** Naive Bayes assumes feature independence, which may underperform on complex linguistic patterns such as sarcasm or context-dependent phrasing.

## XVIII. APPLICATIONS

- Spam filtering for enterprise and consumer email providers.
- Email prioritization for business communication workflows.
- Automated inbox organization and labeling.
- Security monitoring and phishing detection systems.

## XIX. FUTURE ENHANCEMENTS

- **Multi-class classification:** Extend labels to promotional, social, and updates categories.
- **Advanced NLP:** Incorporate Named Entity Recognition (NER), dependency parsing, and contextual embeddings (BERT/RoBERTa).
- **Deep learning models:** Replace Naive Bayes with LSTM or transformer-based classifiers for improved accuracy on complex patterns.
- **Stream processing:** Integrate Apache Kafka and Spark Streaming for real-time classification at scale.
- **Federated learning:** Enable privacy-preserving model updates without centralizing email data.

## XX. CONCLUSION

This paper presents a scalable, end-to-end email categorization system built on Hadoop, MapReduce, and Apache Mahout. All objectives were achieved: emails are ingested into HDFS, preprocessed using a custom MapReduce job, vectorized using TF-IDF, classified using a Naive Bayes model trained in Mahout, and served through an interactive React dashboard with feedback logging. Experimental results demonstrate strong classification performance (F1-score: 99.41%), confirming the viability of this approach for large-scale email analysis. Future work will focus on extending to multi-class scenarios, incorporating deep learning, and enabling real-time stream processing.

## REFERENCES

- [1] Apache Software Foundation, "Apache Hadoop," [Online]. Available: <https://hadoop.apache.org/>. [Accessed: Apr. 2026].
- [2] Apache Software Foundation, "Apache Mahout," [Online]. Available: <https://mahout.apache.org/>. [Accessed: Apr. 2026].
- [3] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in *Proc. 6th Symp. Operating Systems Design and Implementation (OSDI)*, San Francisco, CA, USA, 2004, pp. 137–150.
- [4] Apache SpamAssassin Project, "SpamAssassin Public Corpus," [Online]. Available: <http://spamassassin.apache.org/publiccorpus/>. [Accessed: Apr. 2026].
- [5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.